“Linux File Descriptor and File I/O Management System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

Amrutha Varshini J	**2520080066**
V. Hasith Varma	**2520090125**
Keerthana G	**2520090132**

Under the Esteemed Guidance of

Raghupathi M
Assistant Professor
Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled "**Linux File Descriptor and File I/O Management System**" is a record of bonafide work completed by **Amrutha Varshini J - 2520080066, V. Hasith Varma - 2520090125, and Keerthana G – 2520090132** submitted in partial fulfillment of the requirements for the award of the Bachelor of Technology degree in Computer Science and Information Technology at K L (Deemed to be) University. The work presented in this report describes the team's implementation of a Linux-based file I/O application and has not been copied from another department, university, or institute.

Amrutha Varshini J	**2520080066**
V. Hasith Varma	**2520090125**
Keerthana G	**2520090132**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the project report entitled "**Linux File Descriptor and File I/O Management System**" is a bonafide work carried out and submitted by **Amrutha Varshini J - 2520080066, V. Hasith Varma - 2520090125, and Keerthana G – 2520090132** in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of Computer Science and Information Technology, K L (Deemed to be) University, during the academic year 2026-2027.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr. K. Anusha, and my guide Raghupathi M Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. K Khaja Shareef (CSIT)**, Head of the ***Department for Computer Science and Information Technology*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Amrutha Varshini J 2520080066

V. Hasith Varma 2520090125

Keerthana G 2520090132

Abstract

The Linux File Descriptor and File I/O Management System is a Linux-based command-line application developed in C to demonstrate how an operating system manages files through file descriptors and low-level system calls. The project provides a menu-driven terminal interface through which a user can create or open a file, read data, write data, append content, move the file pointer, display file information, and close an open file. The work focuses on practical Linux/POSIX file handling rather than a graphical, web, or database application.

The implementation uses open(), read(), write(), lseek(), and close() for file operations, together with stat() to retrieve file metadata. The application stores the active file descriptor and current file name, checks whether a file is open before operations that require it, and reports system-call failures using perror(). The append function opens the current file with O_APPEND through a separate descriptor, writes the supplied data, and closes that temporary descriptor.

The project demonstrates the relationship between a user-space C program, Linux system calls, the kernel, and the file system. It also illustrates file offsets, the standard file descriptors, file permissions, file size, file type, and basic error handling. The result is a working terminal application that connects theoretical Operating Systems concepts with practical systems programming using C, GCC, Make, and Ubuntu/Linux.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		8
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		9
		2.1 Files and File Descriptors	
		2.2 System Calls and Kernel Interaction	
		2.3 File Offsets and File Access	
		2.4 Permissions, Metadata and Error Handling	
3	System Requirements		10
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment	
		3.4 Development Tools and Build Configuration	
4	Technology and Tools Used		11
		4.1 C Programming Language	
		4.2 GCC Compiler and Makefile	
		4.3 Linux System Calls and APIs	
		4.4 Linux Commands and Utilities	
		4.5 Git and GitHub Repository	
		4.6 Project Source Structure	
5	System Architecture and Design		12
		5.1-5.3 Overall Architecture, Workflow and Data Flow	
		5.4 Module Description	
		5.5 Input and Output Design	

6	**Implementation**		**13**
		6.1 Project Structure and Main Menu	
		6.2 File Creation and Opening	
		6.3 Read and Write Operations	
		6.4 Append and File-Pointer Operations	
		6.5 Metadata, Closing and Exit	
		6.6 Error Handling	
		6.7 Compilation and Build Process	
		6.8 Implementation Notes and Limitations	
7	**Execution, Testing and Demonstration**		**14**
		7.1 Compilation in Ubuntu / Linux	
		7.2 Execution Commands and Menu	
		7.3 Sample Input and Output	
		7.4 Test Cases and Results	
		7.5 Demonstration of OS Concepts	
8	**Conclusion**		**15**
		8.1 Summary of the Project	
		8.2 Objectives and Results Achieved	
		8.3 Current Limitations	
		8.4 Learning Outcomes and Future Enhancements	
9	**References**		**16**
		9.1 Project Documents and Source Files	
		9.2 Linux / POSIX Manual Pages	
		9.3 Compiler and Build Tool Documentation	
		9.4 Repository and Course Details	

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Files are one of the primary ways in which an operating system stores and retrieves persistent information. In Linux, a process normally accesses an opened file through a file descriptor, which is a non-negative integer returned by the operating system. The process passes this descriptor to system calls such as read() and write() when it needs to transfer data.

At the application level, programmers may use standard C library functions for file handling. However, Operating Systems and Systems Programming also require an understanding of the lower-level interface through which a program requests services from the kernel. This project focuses on that interface and demonstrates file operations directly through Linux/POSIX system calls.

1.2 Problem Statement

File operations are often experienced through familiar applications or high-level library functions, which can hide the role of file descriptors, file offsets, permissions, and kernel-managed resources. As a result, the relationship between a user program and the operating system may remain theoretical. The project addresses this learning gap by developing a terminal-based C application that performs common file operations using Linux system calls and displays relevant information to the user.

1.3 Scope of the Project

The project is limited to a command-line application developed in C and intended to run in a Linux/Ubuntu environment. It supports one primary active file descriptor at a time and provides options to create or open a file, read, write, append, move the file pointer, display file information, close the file, and exit. The append operation temporarily opens the same path using a separate descriptor with the O_APPEND flag.

The application demonstrates file size, permission bits, file type, the current file offset, and basic system-call error reporting. It is an academic systems-programming demonstration. It does not provide a graphical user interface, network file service, database, multi-user access control layer, or a general-purpose file browser.

1.4 Objectives of the Project

- To understand and demonstrate the concept of file descriptors and their role in Linux file management.
- To implement basic file operations using open(), read(), write(), lseek(), and close().
- To demonstrate file metadata, permissions, file offsets, and error handling using Linux/POSIX APIs.
- To develop a working terminal-based system that provides practical understanding of Linux file I/O and the interaction between user programs and the operating system kernel.

1.5 Importance of the Application

The project connects operating-system concepts to visible program behavior. The descriptor printed after opening a file makes it possible to observe that the operating system returns an integer handle. Reading and writing through that handle show how a process requests data transfer. Moving the file offset illustrates how the same file can be accessed from different positions, while stat() demonstrates how metadata can be obtained separately from file content.

Because the program is menu-driven, a learner can repeat operations, observe their effects, and relate each menu choice to the system call used by the implementation. The code is compact enough to inspect while still covering several important file-management concepts.

1.6 Target Users / Audience

The intended audience includes students studying Operating Systems, Linux programming, and C systems programming, as well as users who want a small practical demonstration of low-level file I/O. The system is designed for learning and demonstration rather than production file-management workloads.

CHAPTER -2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Files and File Descriptors

In Linux, an opened file is represented to a process by a file descriptor. The descriptor is a non-negative integer that identifies an open file resource for that process. The kernel maintains the underlying open-file information; the application uses the integer descriptor when invoking operations such as read(), write(), lseek(), and close().

The standard file descriptors are conventionally numbered 0, 1, and 2. Descriptor 0 is standard input, descriptor 1 is standard output, and descriptor 2 is standard error. In a typical terminal process, the first additional file opened by the program may receive descriptor 3, although the exact number depends on which descriptors are already in use.

In this project, the global variable fd stores the descriptor for the currently active file. A value of -1 is used by the application to indicate that no primary file is open. The current file path is stored separately in current_file so that the program can later request metadata or open the same path for appending.

2.2 System Calls and Kernel Interaction

A system call is the controlled interface through which a user-space program requests an operating-system service. The application calls Linux functions such as open(), read(), write(), lseek(), close(), and stat(). The kernel validates the request, applies access rules, performs the required file-system operation, and returns a result or an error indication.

The project flow is: user selection in the terminal, handling by the C application, invocation of a Linux system-call interface, kernel processing, and access to the file system and storage. The application does not directly manipulate the physical storage device. It requests operations through the operating system.

2.3 File Offsets and File Access

An open file description maintains a current file offset that indicates the position at which the next read or write operation takes place. The lseek() system call changes this position. The project demonstrates three reference positions: SEEK_SET (beginning of the file), SEEK_CUR (current position), and SEEK_END (end of the file).

The read function in this application first calls lseek(fd, 0, SEEK_SET), so every read operation begins at the start of the file. The write function also moves the descriptor to the beginning before writing. The separate append descriptor is opened with O_APPEND, which causes writes through that descriptor to be directed to the end of the file.

2.4 Permissions, Metadata and Error Handling

The create/open function supplies permission bits 0644 when a new file is created. These requested permissions are subject to the process umask. The stat() API retrieves information about the named file. The application displays the file size from st_size, the permission bits using st_mode & 0777, and a basic file type classification using S_ISREG() and S_ISDIR().

System-call failures are checked by comparing return values against the documented failure value. When a call fails, perror() displays an error description associated with errno. The program also checks application state before read, write, seek, and close operations so that it can explain when no primary file is currently open.

CHAPTER -3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The project review presentation lists the following minimum hardware requirements: a laptop or desktop computer, at least 4 GB of RAM, a processor rated at a minimum of 1 GHz, and sufficient storage for the Linux environment and project files. These are the requirements stated in the project review material; the application itself is a small command-line C program.

3.2 Software Requirements

- Operating system: Linux / Ubuntu, used as the development and execution environment.
- Programming language: C, used to implement the application and invoke low-level file I/O APIs.
- Compiler: GCC, used to compile src/main.c into the fileio executable.
- Build tool: Make, using the supplied Makefile to automate compilation and cleanup.
- Editor: VS Code or Vim, as listed in the project proposal and review presentation.
- Version control: Git and GitHub, used for source-code versioning and collaboration.

3.3 Operating System Environment

The application is intended to run on Linux/Ubuntu because it uses Linux/POSIX headers and interfaces, including fcntl.h, unistd.h, sys/stat.h, and errno.h. These provide the declarations and definitions required for file opening flags, file descriptors, read/write/seek/close operations, file metadata, and error reporting.

3.4 Development Tools and Build Configuration

The Makefile defines CC as gcc, applies the compiler flags -Wall -Wextra -std=c11, uses src/main.c as the source file, and produces an executable named fileio. The default make target builds the application. The clean target removes the generated executable. No external third-party library is required by the supplied source code.

CHAPTER -4 TECHNOLOGY AND TOOLS USED

4.1 C Programming Language

C is used because it provides direct access to the operating-system interfaces required by this project. The source file uses standard C facilities for input, output, string length, and program control, along with Linux/POSIX APIs for file operations. The application defines a 1024-byte buffer for data transfer and uses bounded input for the file name.

4.2 GCC Compiler and Makefile

GCC compiles the source file src/main.c into the executable fileio. The Makefile groups the compiler, flags, source path, and target name so the build can be repeated with the make command. The flags -Wall and -Wextra request common compiler warnings, while -std=c11 selects the C11 language standard.

4.3 Linux System Calls and APIs

The following interfaces form the core of the implementation:

- open(): opens an existing file or creates a new file when O_CREAT is supplied. It returns a file descriptor on success and -1 on failure.
- read(): transfers bytes from the file into a user-space buffer. The program reads up to BUFFER_SIZE - 1 bytes in one call and adds a terminating null character for display.
- write(): transfers the supplied bytes from the application buffer to the file. The number of bytes reported as written is displayed.
- lseek(): changes the file offset. The program uses it to reset the position for read/write and to demonstrate SEEK_SET, SEEK_CUR, and SEEK_END.
- close(): releases an open file descriptor. The primary descriptor is reset to -1 after a successful close.
- stat(): retrieves metadata for the current file path, including size, permissions, and file type.
- errno / perror(): provides an error description when a checked system call fails.

4.4 Linux Commands and Utilities

The main commands used to build and run the project are make, ./fileio, and make clean. The application is operated through the menu displayed in the terminal. Standard Linux file inspection commands such as ls, cat, and stat may also be used during manual verification, but they are not invoked internally by the C application.

4.5 Git and GitHub Repository

Git is used for version control and GitHub hosts the team repository. The project problem statement records the repository at https://github.com/amrutha-0123/KLH-CSIT-AIDS-2026-27-Team4-FDFM. The repository contains the C source, Makefile, README, project documents, and test data.

4.6 Project Source Structure

The main implementation is located in src/main.c. The Makefile is at the project root. README.md describes the project, its features, build steps, sample output, tests, learning outcomes, and team contributions. The supplied repository also contains data and documentation folders and the built fileio executable.

CHAPTER -5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The application follows a layered interaction model. A user selects an operation through the terminal menu. The C application validates its current state and calls the relevant Linux interface. The kernel manages the open-file resource and performs the requested operation through the file system. Results and error messages are then returned to the terminal.

User Input → Terminal Menu → C Application → Linux System Call → Linux Kernel → File System → Storage

The application layer contains the menu and operation functions. The system-call layer includes open(), read(), write(), lseek(), close(), and stat(). The kernel and file system provide the operating-system services; the application requests those services rather than accessing storage directly.

5.2 System Workflow

When the program starts, it initializes the primary descriptor to -1 and the current file path to an empty string. It displays the menu repeatedly until the user chooses Exit. For Create/Open, the program closes a previously active primary descriptor, if any, then calls open() with O_RDWR | O_CREAT and permission argument 0644. On success, it saves the returned descriptor and the entered path.

Read and Write require the primary descriptor to be valid. Read resets the offset to the beginning, reads one buffer-sized block, and displays the returned bytes. Write collects a line of text, moves to the beginning, and writes the line. Append checks that a path has been recorded, opens that path with O_WRONLY | O_APPEND, writes the entered line, and closes the temporary descriptor.

Move File Pointer asks for an offset and one of three reference positions, then calls lseek(). Display File Information calls stat() using the stored path. Close releases the primary descriptor. Exit closes the primary descriptor if it remains open before terminating.

5.3 Data Flow and Operation Sequence

The main data path for a read or write operation is between the file descriptor, the application buffer, and the kernel-mediated file resource. For reads, bytes are copied from the file into the buffer. For writes, bytes from the buffer are passed to the operating system. The append function uses a second descriptor whose O_APPEND flag directs each write to the end of the file.

The metadata path is separate from the content-transfer path: the application supplies the file name to stat(), receives a struct stat, and displays selected fields. This makes it possible to show the distinction between a file path, an open file descriptor, file content, and file metadata.

5.4 Module Description

- Create/Open module: reads a file name, closes any prior primary descriptor, opens or creates the requested file, and stores the descriptor and path.
- Read module: verifies the descriptor, resets the offset, reads a limited block, and displays the content and byte count.
- Write module: collects input, resets the offset to the beginning, writes the entered bytes, and reports the count.
- Append module: opens the stored path with O_APPEND, writes the entered bytes using a separate descriptor, and closes it.
- Pointer module: uses lseek() to set the offset relative to the beginning, current position, or end of the file.
- Information module: calls stat() and displays file name, size, permissions, and file type.
- Close and Exit modules: close the active descriptor and update the program state.

5.5 Input and Output Design

Inputs are entered through the terminal. The menu accepts an integer choice. Create/Open accepts a file name, Write and Append accept a line of text, and Move File Pointer accepts an offset and a reference-position choice. Outputs include success messages, the descriptor value, bytes read or written, the new file offset, metadata fields, and error descriptions. The code limits the file-name input to 255 characters and uses a 1024-byte data buffer.

CHAPTER -6 IMPLEMENTATION

6.1 Project Structure and Main Menu

The repository organizes the implementation under src/main.c and keeps the Makefile and README.md at the project root. The source includes stdio.h, stdlib.h, fcntl.h, unistd.h, sys/stat.h, errno.h, and string.h. The main() function prints the application heading and enters a loop that displays eight menu options and dispatches the selected option through a switch statement.

The two shared state variables are int fd = -1 and char current_file[256] = "". The first tracks the currently open primary descriptor; the second stores the file path for metadata and append operations. BUFFER_SIZE is defined as 1024.

6.2 File Creation and Opening

The create_open_file() function prompts for a file name and reads it using a bounded %255s conversion. If another primary file descriptor is active, it closes that descriptor and sets fd to -1. It then calls open(filename, O_RDWR | O_CREAT, 0644). If the call fails, perror() reports the reason. If successful, the function copies the path into current_file and displays the descriptor returned by the operating system.

The O_RDWR flag permits both reading and writing through the descriptor. O_CREAT requests creation when the path does not already exist. The mode 0644 requests read/write permission for the owner and read permission for group and others, subject to the process umask.

6.3 Read and Write Operations

The read_file() function first checks whether fd equals -1. If no primary file is open, it prints a message and returns. Otherwise, it calls lseek(fd, 0, SEEK_SET), reads up to BUFFER_SIZE - 1 bytes into a local buffer, checks for a read error, adds a null terminator, and prints either an empty-file message or the content. It also displays the byte count returned by read().

The write_file() function similarly checks that a file is open. It obtains a line with fgets(), moves the file offset to the beginning using lseek(), and calls write() with strlen(data) bytes. A successful write is followed by a message showing the number of bytes written. Because the function does not call ftruncate(), writing shorter content over a longer existing file may leave old bytes after the newly written data.

6.4 Append and File-Pointer Operations

The append_file() function uses current_file to reopen the file with O_WRONLY | O_APPEND. This creates a separate temporary descriptor, append_fd. The function reads a line of input, writes it through append_fd, and closes that descriptor. If the write fails, it reports the error and attempts to close the temporary descriptor before returning.

The move_file_pointer() function accepts an offset and a choice. Choice 1 calls lseek(fd, offset, SEEK_SET), choice 2 uses SEEK_CUR, and choice 3 uses SEEK_END. It checks for the failure value (off_t)-1 and, on success, prints the resulting file position.

6.5 Metadata, Closing and Exit

The display_file_info() function checks whether a file path has been recorded and then calls stat(current_file, &file_info). It displays the file name, file size in bytes, permission bits masked with 0777, and a file-type label. The file type is classified as Regular file, Directory, or Other using the corresponding S_IS* macros.

The close_file() function checks the primary descriptor, calls close(fd), and sets fd to -1 after a successful close. The current_file string is not cleared by this function, so metadata lookup by path can still be requested after the primary descriptor has been closed, provided the file continues to exist. When the user selects Exit, main() closes fd if it is still open and then returns.

6.6 Error Handling

The source checks return values from open(), read(), write(), lseek(), stat(), and close() where those operations are used. perror() is called for several system-call failures so the terminal can display the reason associated with errno. The application also checks whether a primary descriptor exists before functions that require it and handles invalid menu choices and invalid file-pointer reference choices.

6.7 Compilation and Build Process

The Makefile sets CC = gcc, CFLAGS = -Wall -Wextra -std=c11, TARGET = fileio, and SOURCE = src/main.c. The all target compiles the source into the fileio executable. The clean target runs rm -f fileio so the generated executable can be removed before a fresh build.

6.8 Implementation Notes and Limitations

Current limitations include one tracked primary descriptor, a 1023-byte read limit, no truncation before overwrite, whitespace-delimited file names, and limited validation of scanf() input.

CHAPTER -7 EXECUTION, TESTING AND DEMONSTRATION

7.1 Compilation in Ubuntu / Linux

Open a terminal in the project root directory, where the Makefile is located. The default build can be started with make. The supplied Makefile invokes GCC with the warning flags and C11 standard and produces an executable named fileio. A successful build allows the program to be launched from the same directory.

The project source was compiled in the preparation environment using the supplied Makefile. The command completed successfully and produced the fileio executable without compiler diagnostics.

7.2 Execution Commands and Menu

The normal execution sequence is to enter the project directory, run make, and then run ./fileio. To remove the generated executable, use make clean. When launched, the program displays the title “Linux File Descriptor and File I/O System” and a menu with Create/Open File, Read File, Write File, Append to File, Move File Pointer, Display File Information, Close File, and Exit.

7.3 Sample Input and Output

A typical demonstration begins by selecting Create/Open File and entering sample.txt. On a successful open, the application prints “File opened successfully!” and the descriptor value returned by Linux. Selecting Write File prompts for a line of data and displays “Data written successfully!” with the number of bytes written. Selecting Read File displays the file content and the number of bytes read.

The Display File Information option prints the stored file name, file size, permission bits, and file type. For a newly created regular file with the requested mode and a typical umask, the displayed permission bits may be 644. The actual descriptor and metadata depend on the execution environment and file state.

7.4 Test Cases and Results

The following functional checks are suitable for demonstrating the implemented features. A successful build and a basic create, write, read, and metadata sequence were executed during preparation of this report. Other cases below are included as functional test cases for review and can be repeated in the project environment.

- Create/open a new file: verify the success message and returned descriptor.
- Write a line of data and read it back: verify the displayed content and byte counts.
- Append a line: verify that the new data is written through the O_APPEND descriptor at the end of the existing file.
- Move the file pointer with SEEK_SET, SEEK_CUR, and SEEK_END: verify that the program prints the resulting offset.
- Display file information: verify the file name, size, permissions, and regular-file classification.
- Close the file and then attempt Read or Write: verify that the application reports that no file is currently open.
- Enter an invalid menu choice or invalid pointer-position choice: verify that the corresponding invalid-choice message is displayed.
- Attempt to open a path that cannot be accessed: verify that open() fails and perror() reports the error.

7.5 Demonstration of OS Concepts

The file descriptor printed by Create/Open demonstrates the integer handle assigned to an opened file. Read and Write demonstrate kernel-mediated data transfer using that descriptor. The pointer menu demonstrates file offsets and the three lseek() reference positions. Append demonstrates O_APPEND with a separate descriptor. Display File Information demonstrates metadata retrieval through stat(), while the error paths demonstrate how the application responds when an operation fails or the required descriptor is not available.

CHAPTER -8 CONCLUSION

8.1 Summary of the Project

The Linux File Descriptor and File I/O Management System is a terminal-based C application that demonstrates low-level file management in Linux. It provides menu options for opening or creating a file, reading, writing, appending, moving the file pointer, displaying metadata, closing the file, and exiting.

8.2 Objectives and Results Achieved

The implementation uses the principal Linux/POSIX interfaces identified in the project proposal: open(), read(), write(), lseek(), close(), and stat(). It maintains a primary file descriptor, demonstrates the use of O_APPEND for appending, displays file size and permission information, and reports several system-call failures. The supplied Makefile builds the application with GCC under the C11 standard.

8.3 Current Limitations

The current implementation is intentionally small and focused on demonstration. It reads a single block per read operation, does not truncate files before overwriting from the beginning, accepts file names without spaces, and does not perform comprehensive validation of every console input. These behaviors are documented so that users can understand the current scope of the implementation.

8.4 Learning Outcomes and Future Enhancements

The project provides practical experience with file descriptors, system calls, file offsets, access flags, file metadata, permissions, and error handling. Further versions could add a loop for reading files of any size, support safe truncation or explicit overwrite modes, validate all input, display the current offset after each operation, and provide more detailed permission and error explanations. Such improvements would extend the demonstration while retaining its focus on Linux file I/O.

Team Contributions. The responsibilities recorded in the project submission are:

- Amrutha Varshini J: overall system design and integration, file creation and opening, and file descriptor management.
- V. Hasith Varma: read, write, append, and file-pointer operations using Linux system calls, and testing of file I/O functionality.
- Keerthana G: file metadata, permissions, and error handling, together with test cases, documentation, and output verification.

CHAPTER -9 REFERENCES

9.1 Project Documents and Source Files

- Linux File Descriptor and File I/O Management System - Project Problem Statement Submission Form, Operating Systems and Systems Programming (25CS2104E), 2026-27, Term-I.
- Linux File Descriptor and File I/O Management System - Review-I presentation, dated 11 September 2026.
- Team repository files: README.md, src/main.c, and Makefile, KLH-CSIT-AIDS-2026-27-Team4-FDFM.

9.2 Linux / POSIX Manual Pages

- open(2), read(2), write(2), lseek(2), close(2), stat(2), and errno(3), Linux manual pages.
- Linux man-pages project: https://man7.org/linux/man-pages/

9.3 Compiler and Build Tool Documentation

- GNU Compiler Collection (GCC) documentation: https://gcc.gnu.org/onlinedocs/
- GNU Make manual: https://www.gnu.org/software/make/manual/

9.4 Repository and Course Details

- GitHub repository: https://github.com/amrutha-0123/KLH-CSIT-AIDS-2026-27-Team4-FDFM

Course: Operating Systems and Systems Programming (25CS2104E), Academic Year 2026-27, Term-I. Section 12, Team 4.